

Project Corpus Forge

RAG-Based Document AI Platform

Generative AI for Software Engineering — EPITA 2026

Team: Chloé · Stanislaw · Victor

Upload → Retrieve → Generate

Problem & Motivation

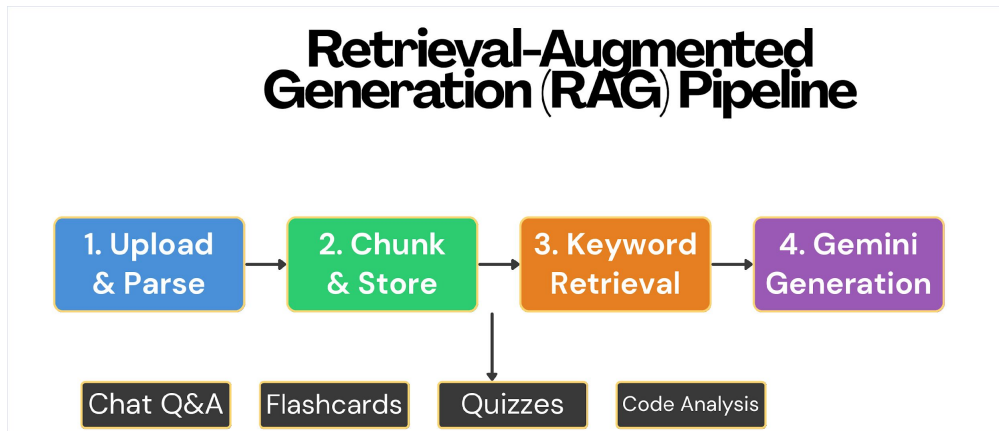
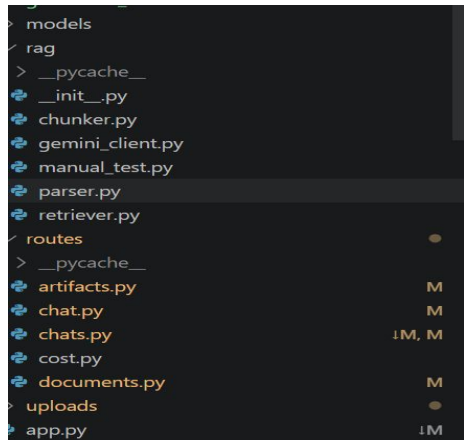
- Students and software teams collect PDFs, notes, and source-code files.
- It is hard to turn these files into study material or engineering summaries quickly.
- Corpus Forge transforms a small corpus into answers, flashcards, quizzes, and code-analysis reports.

Platform Overview

- Upload documents and source-code files.
- Ask grounded questions using retrieved context.
- Generate flashcards and quizzes.
- Generate source-code analysis reports.
- Track estimated AI usage and token cost.

Architecture / RAG Pipeline

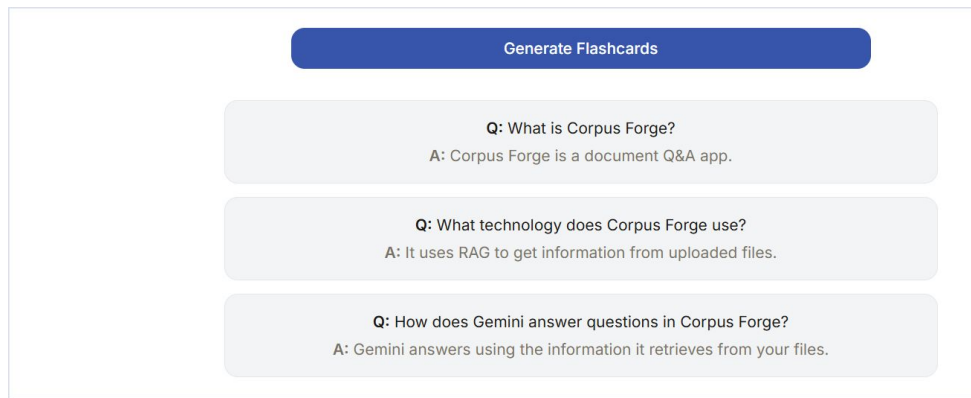
- React frontend sends requests to Flask.
- Flask handles upload, parsing, chunking, retrieval, Gemini calls, and persistence.
- SQLite stores documents, chunks, artifacts, chats, and cost records.
- Gemini receives prompts grounded in retrieved chunks.



RAG pipeline: upload, parse, chunk, retrieve, prompt, generate.

Flashcard Generation

- Retrieves relevant document chunks.
- Sends grounded prompt to Gemini.
- Displays generated cards when JSON is valid.
- Falls back to raw text if parsing fails.



The screenshot displays a user interface for generating flashcards. At the top, there is a blue button labeled "Generate Flashcards". Below this button, three flashcards are shown, each with a question (Q) and an answer (A). The first flashcard asks "Q: What is Corpus Forge?" and answers "A: Corpus Forge is a document Q&A app." The second flashcard asks "Q: What technology does Corpus Forge use?" and answers "A: It uses RAG to get information from uploaded files." The third flashcard asks "Q: How does Gemini answer questions in Corpus Forge?" and answers "A: Gemini answers using the information it retrieves from your files."

Generate Flashcards

Q: What is Corpus Forge?
A: Corpus Forge is a document Q&A app.

Q: What technology does Corpus Forge use?
A: It uses RAG to get information from uploaded files.

Q: How does Gemini answer questions in Corpus Forge?
A: Gemini answers using the information it retrieves from your files.

Generated flashcards from retrieved document context.

Quiz Generation

- Generates questions from corpus context.
- Can include options, answers, and explanations.
- Saves artifact metadata for persistence.

The screenshot displays a web interface for generating a quiz. At the top, there is a text input field containing 'Corpus Forge' and a numeric input field with the value '3'. Below these is a blue button labeled 'Generate Quiz'. The main content area shows two quiz questions. The first question, 'Q1: What is Corpus Forge?', has four multiple-choice options: 'A document Q&A app', 'A code editor', 'A video editing software', and 'A music player'. Below the options, the correct answer is displayed: 'Answer: A — The document states that Corpus Forge is a document Q&A app.' The second question, 'Q2: What technology does Corpus Forge use to find information in uploaded files?', has four multiple-choice options: 'RAG', 'AI', 'Machine Learning', and 'Blockchain'.

Quiz questions generated from the uploaded corpus.

Source-Code Analysis

- Supports .py and .js uploads.
- Produces code review comments.
- Explains architecture and control flow.
- Suggests bugs and improvements.

Code Analysis

Generate a code review report from your uploaded source files.

calculate_total print_invoice code review architecture control flow

Generate Code Analysis

Code Review Report

Code Review Comments:

- The `'calculate_total'` function correctly iterates through a list of items and sums their prices.
 - The `'apply_discount'` function correctly calculates the discounted total.
- The `'print_invoice'` function orchestrates the invoice generation by calling `'calculate_total'` and `'apply_discount'` and then printing the details.
 - The provided example usage for `'print_invoice'` is clear and demonstrates its functionality.

Possible Bugs:

- No obvious bugs are present in the provided code snippets for `'calculate_total'`, `'apply_discount'`, and `'print_invoice'`. The logic appears sound for the intended purpose.

Architecture Overview:

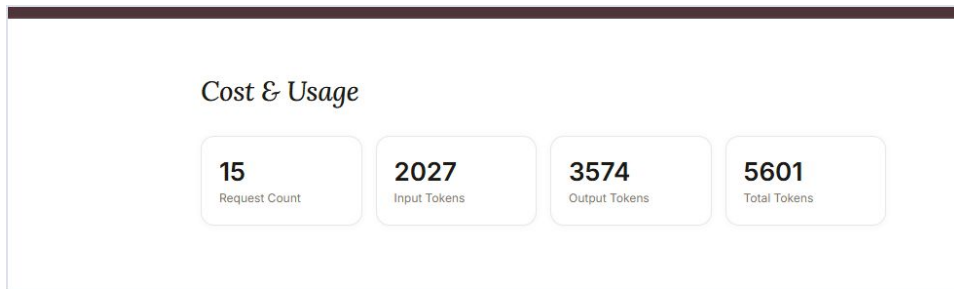
The code defines a simple architecture for generating an invoice. It consists of three main functions:

1. `'calculate_total'`: Responsible for calculating the sum of item prices.
2. `'apply_discount'`: Responsible for applying a percentage-based discount to a given total.
3. `'print_invoice'`: Acts as the main orchestrator, using the other two functions to compute the

Code review, architecture, control-flow, bugs, and improvement suggestions.

Cost Observability & Persistence

- Uploaded documents and chunks persist in SQLite.
- Generated artifacts are saved locally and tracked in the database.
- Backend records request count and estimated input/output tokens.
- Cost page displays total usage.



Estimated request count and token usage tracked by the backend.

Engineering Challenges

- Frontend was initially mock-only and had to be wired to the real Flask backend.
- Retrieval initially risked sending unrelated chunks, so only positive-score chunks are returned.
- Gemini quota/rate limits required shorter prompts and fewer retrieved chunks.
- Active document selection exists visually; backend filtering by selected documents is future work.

AI Collaboration

- GitHub Copilot helped with implementation.
- ChatGPT/Codex-style review helped with planning, debugging, and documentation.
- AI suggestions were tested and corrected, not accepted blindly.
- We used AI as an engineering partner, not as a shortcut.

Limitations & Future Improvements

- Add vector search and embeddings.
- Filter retrieval by selected active documents.
- Add document deletion and full corpus management.
- Improve code-analysis report rendering.
- Add automated backend and frontend tests.
- Use exact Gemini token metadata when available.

Live Demo Plan / Conclusion

Demo flow

- Upload a document or source-code file.
- Ask a grounded question.
- Generate flashcards.
- Generate a quiz.
- Run code analysis.
- Show cost tracking.

Conclusion

Corpus Forge shows how RAG can transform heterogeneous documents into actionable knowledge while demonstrating practical engineering decisions, persistence, reliability, and honest limitations.